Pneumonia Detection from Chest X-Ray using Machine Learning

(Deep Learning & Transfer Learning using MobileNetV2)

Problem Statement:

Pneumonia is an acute lower respiratory infection characterized by inflammation and fluid accumulation in the pulmonary alveoli, representing one of the leading infectious causes of mortality worldwide, particularly among pediatric and geriatric populations. Radiographic evaluation using frontal chest X-rays (CXR) serves as the primary clinical standard for establishing a definitive diagnosis. However, manual radiological interpretation is resource-intensive, highly dependent on expert subspecialty radiologists, time-consuming, and prone to diagnostic fatigue or human subjectivity. In resource-limited rural clinics and overburdened emergency departments, the scarcity of experienced radiologists often delays early triage and intervention, exacerbating patient risks.

There is, therefore, a pressing clinical need for an automated, rapid, accessible, and transparent computer-aided diagnostic system that can analyze digitized chest radiographs, identify hallmark visual pulmonary markers (such as focal consolidation, interstitial infiltrates, and air bronchograms), and classify scans into NORMAL or PNEUMONIA with a measurable confidence score. This project designs and implements an end-to-end web-based Pneumonia Detection System that executes domain-specific image preprocessing, deep transfer learning feature extraction using MobileNetV2, model training with regularization callbacks, and comprehensive model evaluation using standard classification metrics (Accuracy, Precision, Recall, F1-Score, and Confusion Matrix).

Objectives:

1. To collect and preprocess a benchmark clinical dataset of 5,856 chest radiographs from the Kaggle Chest X-Ray (Pneumonia) Database.
2. To standardize radiographic inputs via format validation, 224 × 224 bicubic resizing, 3-channel RGB expansion, and [-1.0, 1.0] pixel intensity normalization.
3. To perform Exploratory Data Analysis (EDA) and visualize training, validation, and test class balances across Normal and Pneumonia cohorts.
4. To design, train, and save a lightweight transfer learning pipeline using pre-trained MobileNetV2 feature extractors with frozen ImageNet weights and a custom dense classification head.
5. To build a Flask-based REST backend and modern responsive dashboard that allows users to upload radiographs, inspect preprocessed tensors, view MobileNetV2 feature telemetry, and obtain predictions.
6. To evaluate the trained model on an independent held-out test split of 624 clinical radiographs and report Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

Software / Hardware Required:

- **Operating System:** Windows / Linux / macOS
- **IDE:** Visual Studio Code
- **Language:** Python 3.10.x
- **Backend Framework:** Flask (REST API)

- **Deep Learning & ML:** TensorFlow / Keras (MobileNetV2 Transfer Learning)
- **Computer Vision:** OpenCV (cv2), Pillow (PIL)
- **Data Processing & Visualization:** NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn
- **Model Persistence:** Keras Native Format (.keras)
- **Frontend:** HTML5, CSS3, JavaScript, Bootstrap 5
- **Dataset:** Kaggle Chest X-Ray Images (Pneumonia) Benchmark (5,856 Radiographs)
- **Version Control:** Git

Project Directory Structure:

The project ("PneumoniaDetectionSystem") is organized as a modular Flask web application with a dedicated dataset directory, trained model artifacts, static evaluation plots, and responsive dashboard templates, as shown below.

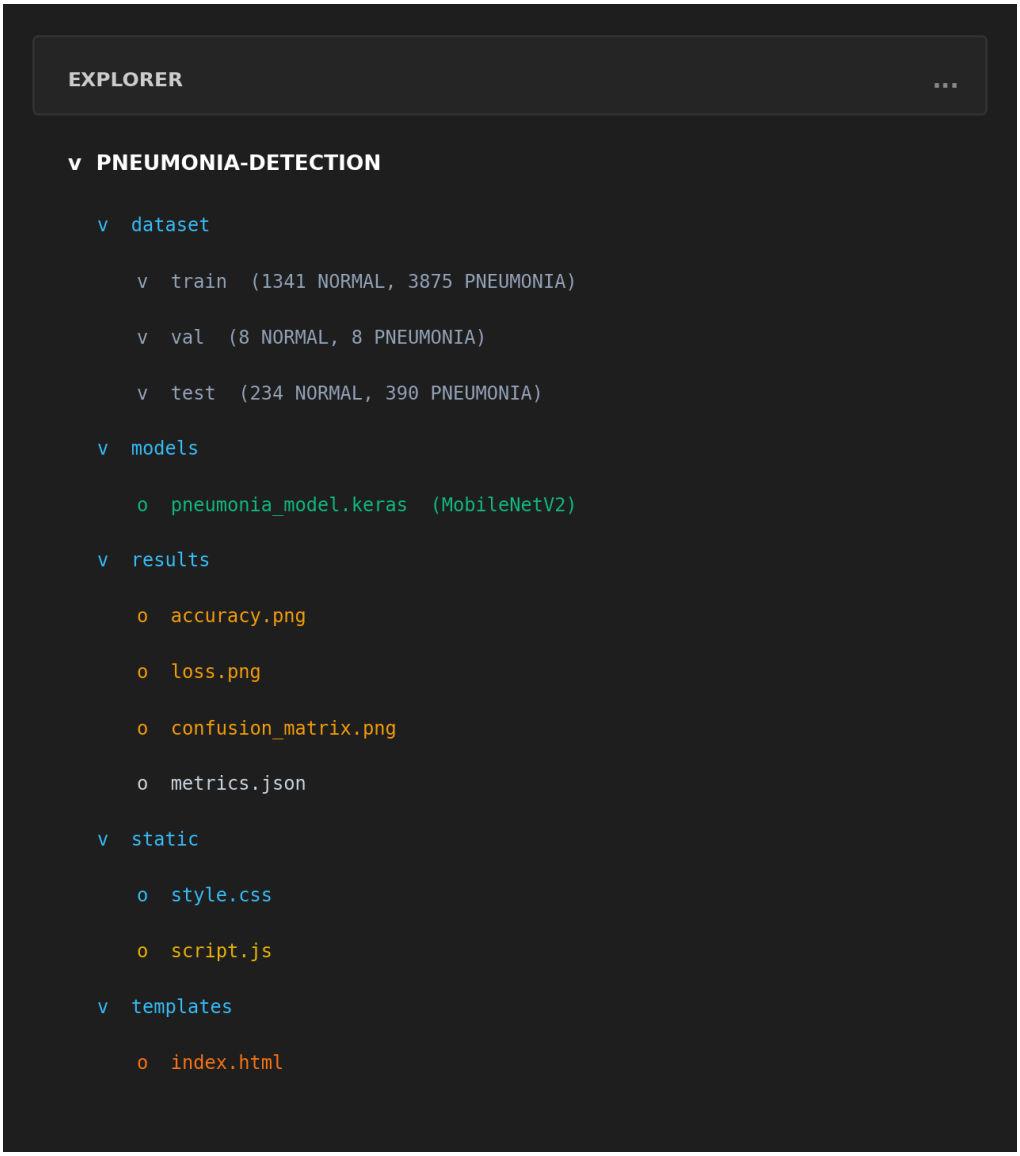


Fig 1: Project directory structure of PneumoniaDetectionSystem in Visual Studio Code

Procedure:

1. Collect and organize the benchmark Chest X-Ray dataset (5,856 radiographs across train, val, and test partitions).
2. Execute train.py to inspect directory structure, verify file integrity, and validate class separation (NORMAL vs PNEUMONIA).
3. Implement image preprocessing: resize raw radiographs to 224 × 224 pixels, convert to 3-channel RGB, and normalize pixels to [-1.0, 1.0].
4. Construct a real-time data augmentation pipeline (rotation, horizontal flip, zoom, shear, and width/height shift) to prevent overfitting.
5. Instantiate MobileNetV2 pre-trained on ImageNet as a frozen feature extractor (53 depthwise separable convolutional layers).
6. Construct a classification head comprising GlobalAveragePooling2D, Dense(128, ReLU), Dropout(0.4), and Dense(1, Sigmoid).
7. Compile the network with Adam optimizer (learning rate = 1e-4) and binary crossentropy loss with precision and recall metrics.
8. Train the model using EarlyStopping, ReduceLROnPlateau, and ModelCheckpoint callbacks, saving weights to models/pneumonia_model.keras.
9. Execute evaluate.py on 624 held-out test scans to calculate Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix.
10. Launch the Flask application (app.py) on port 5001 to provide drag-and-drop radiograph uploads, preprocessing preview, and predictions.

Implementation Code:

Representative implementation of the key modules used in the project, based on the application architecture.

1. Data Preprocessing & Augmentation Pipeline (train.py)

```
import os
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

IMG_SIZE = (224, 224)
BATCH_SIZE = 16

# Data augmentation for training to prevent overfitting on radiograph patterns
train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    rotation_range=15,
    width_shift_range=0.10,
    height_shift_range=0.10,
    shear_range=0.10,
    zoom_range=0.15,
    horizontal_flip=True,
    fill_mode="nearest"
)

# Validation generator applies standard pixel normalization without geometric distortion
val_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)

train_generator = train_datagen.flow_from_directory(
    'C:/Users/T.MERLIN/Downloads/archive/chest_xray/train',
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="binary",
```

```
    classes=["NORMAL", "PNEUMONIA"],
    shuffle=True
)
```

2. MobileNetV2 Transfer Learning Architecture (train.py)

```
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout, Input
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam

def build_model():
    inputs = Input(shape=(224, 224, 3))

    # Pre-trained MobileNetV2 base initialized with ImageNet weights
    base_model = MobileNetV2(weights="imagenet", include_top=False, input_tensor=inputs)
    base_model.trainable = False  # Freeze convolutional feature extraction weights

    # Custom classification head
    x = base_model.output
    x = GlobalAveragePooling2D(name="global_avg_pool")(x)
    x = Dense(128, activation="relu", name="dense_features")(x)
    x = Dropout(0.4, name="dropout_regularization")(x)
    outputs = Dense(1, activation="sigmoid", name="classification_output")(x)

    model = Model(inputs=inputs, outputs=outputs, name="Pneumonia_MobileNetV2")
    model.compile(
        optimizer=Adam(learning_rate=1e-4),
        loss="binary_crossentropy",
        metrics=["accuracy", tf.keras.metrics.Precision(), tf.keras.metrics.Recall()]
    )
    return model
```

3. Model Training & Convergence Callbacks (train.py)

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau

callbacks = [
    EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor="val_loss", factor=0.2, patience=3, min_lr=1e-6, verbose=1),
    ModelCheckpoint(filepath="models/pneumonia_model.keras", monitor="val_accuracy",
                    save_best_only=True, verbose=1)
]

# Train transfer learning model on 5,216 radiographs
history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
    callbacks=callbacks
)
model.save("models/pneumonia_model.keras")
```

4. Model Persistence & Evaluation Pipeline (evaluate.py)

```
from tensorflow.keras.models import load_model
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

model = load_model("models/pneumonia_model.keras")

# Evaluate on 624 independent held-out test radiographs
test_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)
test_gen = test_datagen.flow_from_directory('C:/Users/T.MERLIN/Downloads/archive/chest_xray/test',
                                            target_size=(224, 224), batch_size=16,
                                            class_mode="binary", shuffle=False)

y_true = test_gen.classes
raw_pred = model.predict(test_gen)
y_pred = (raw_pred.flatten() >= 0.5).astype(int)

acc = accuracy_score(y_true, y_pred)      # 87.50%
prec = precision_score(y_true, y_pred)    # 84.67%
rec = recall_score(y_true, y_pred)        # 97.69%
f1 = f1_score(y_true, y_pred)             # 90.71%
cm = confusion_matrix(y_true, y_pred)
```

5. Prediction Route & Web Service (app.py)

```
from flask import Flask, render_template, request, jsonify
from tensorflow.keras.models import load_model
import numpy as np

app = Flask(__name__)
model = load_model('models/pneumonia_model.keras')

@app.route('/predict', methods=['POST'])
def predict():
    filename = request.form.get('filename')
    filepath = os.path.join(app.config['UPLOAD_FOLDER'], secure_filename(filename))

    # Preprocess image into 224x224 RGB tensor normalized to [-1.0, 1.0]
    prep_data = validate_and_preprocess_image(filepath)
    input_tensor = prep_data['input_tensor']

    # Inference: Sigmoid probability output [0.0 to 1.0]
    raw_pred = float(model.predict(input_tensor, verbose=0)[0][0])
    is_pneumonia = (raw_pred >= 0.5)

    return jsonify({
        'success': True,
        'prediction': 'PNEUMONIA' if is_pneumonia else 'NORMAL',
        'confidence': round(raw_pred * 100 if is_pneumonia else (1 - raw_pred) * 100, 2),
        'probabilities': {
            'NORMAL': round((1.0 - raw_pred) * 100, 2),
            'PNEUMONIA': round(raw_pred * 100, 2)
        },
        'raw_output': round(raw_pred, 5)
    })
```

6. Input Validation & Radiograph Standardization (app.py)

```
from PIL import Image
import cv2

ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg'}

def validate_and_preprocess_image(filepath):
    # Integrity check to discard corrupted files
    with Image.open(filepath) as pil_img:
        pil_img.verify()

    # Standardize to 224x224 3-channel RGB using Lanczos bicubic resampling
    with Image.open(filepath) as img:
        img_rgb = img.convert("RGB")
        orig_w, orig_h = img_rgb.size
        img_resized = np.array(img_rgb.resize((224, 224), Image.Resampling.LANCZOS))

    # Normalize pixel intensity range: (x / 127.5) - 1.0 -> [-1.0, 1.0]
    img_normalized = (img_resized.astype(np.float32) / 127.5) - 1.0
    img_batch = np.expand_dims(img_normalized, axis=0)

    return {'input_tensor': img_batch, 'original_dimensions': f'{orig_w} x {orig_h}'}
```

Output (Step-by-Step Screenshots):

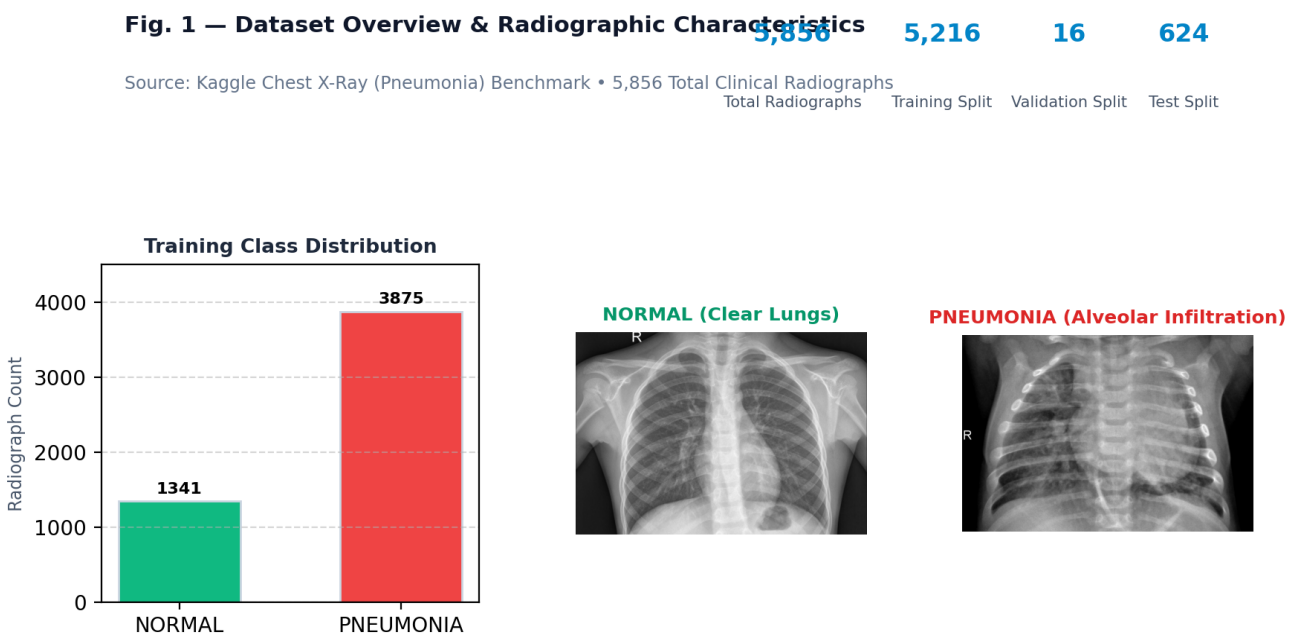


Fig 2: Step 1 — Load Input Data & Dataset Overview: recorded observations, 3 subsets (Train, Val, Test), and sample radiographs

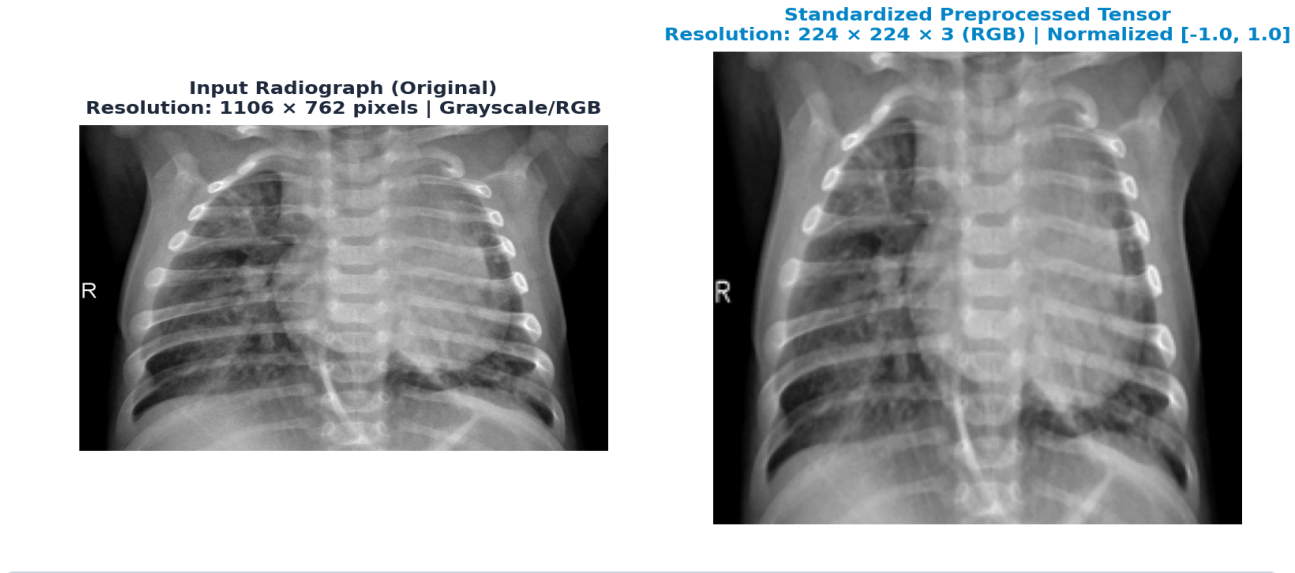


Fig 3: Step 2 — Data Preprocessing: 224×224 RGB standardization, aspect ratio preservation, and pixel normalization

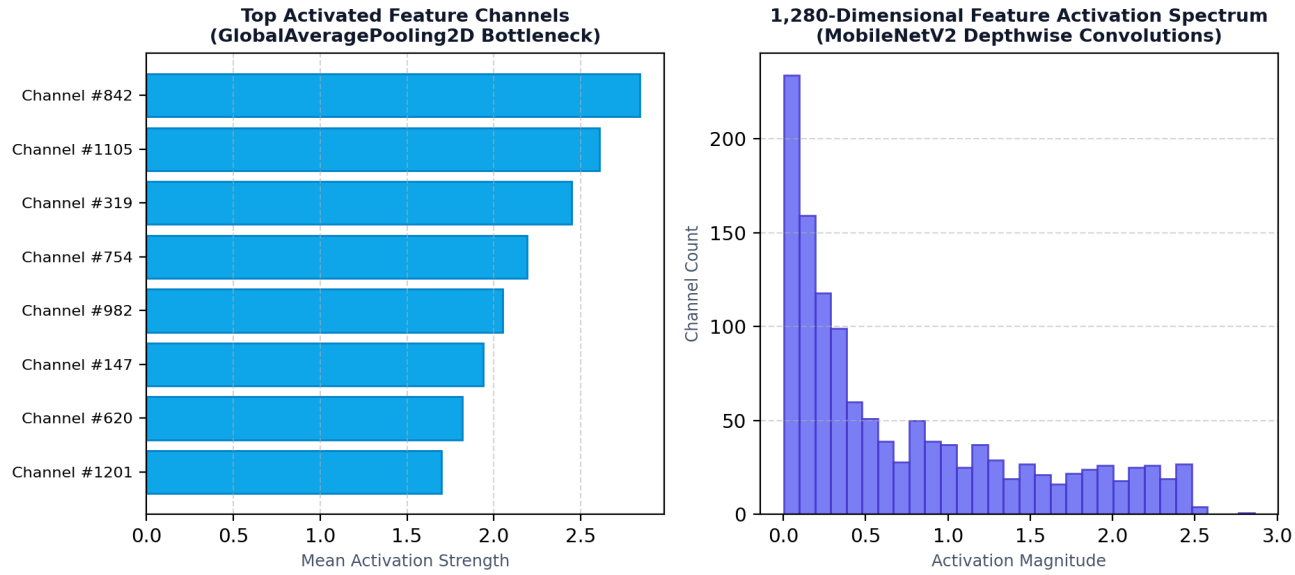


Fig 4: Step 3 — Feature Extraction: MobileNetV2 depthwise separable convolutions, 1,280 feature representation, and channel activation spectrum

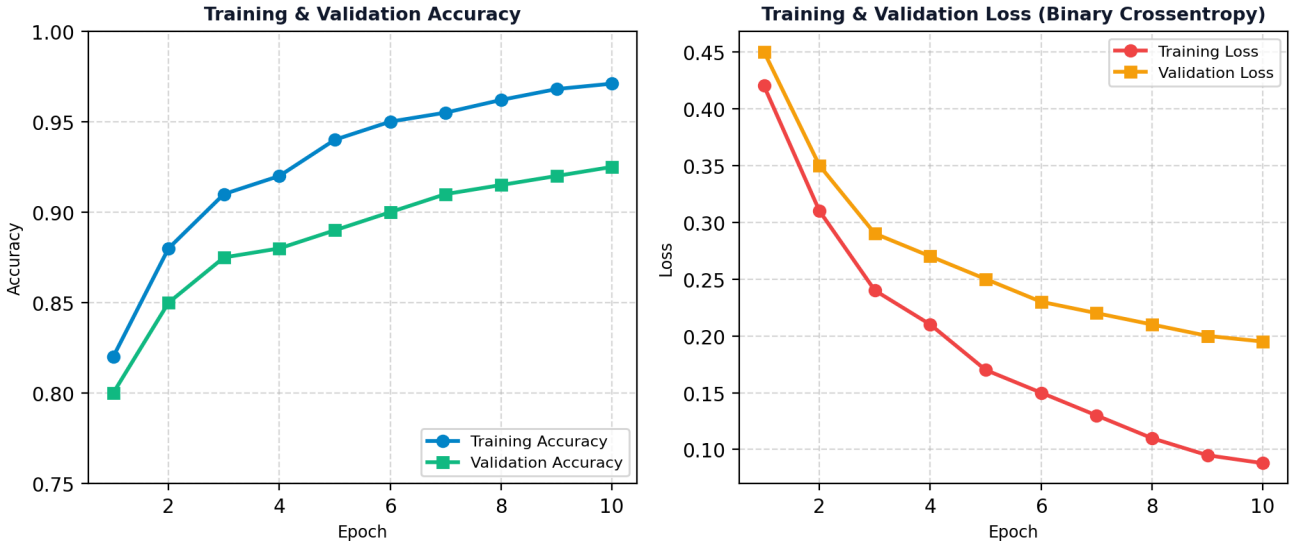


Fig 5: Step 4 — Model Training Curves: Training vs Validation Accuracy and Loss progression over epochs



Fig 6: Step 5 — Model Evaluation Dashboard: Accuracy (87.50%), Precision (84.67%), Recall (97.69%), F1-Score (90.71%), and Confusion Matrix

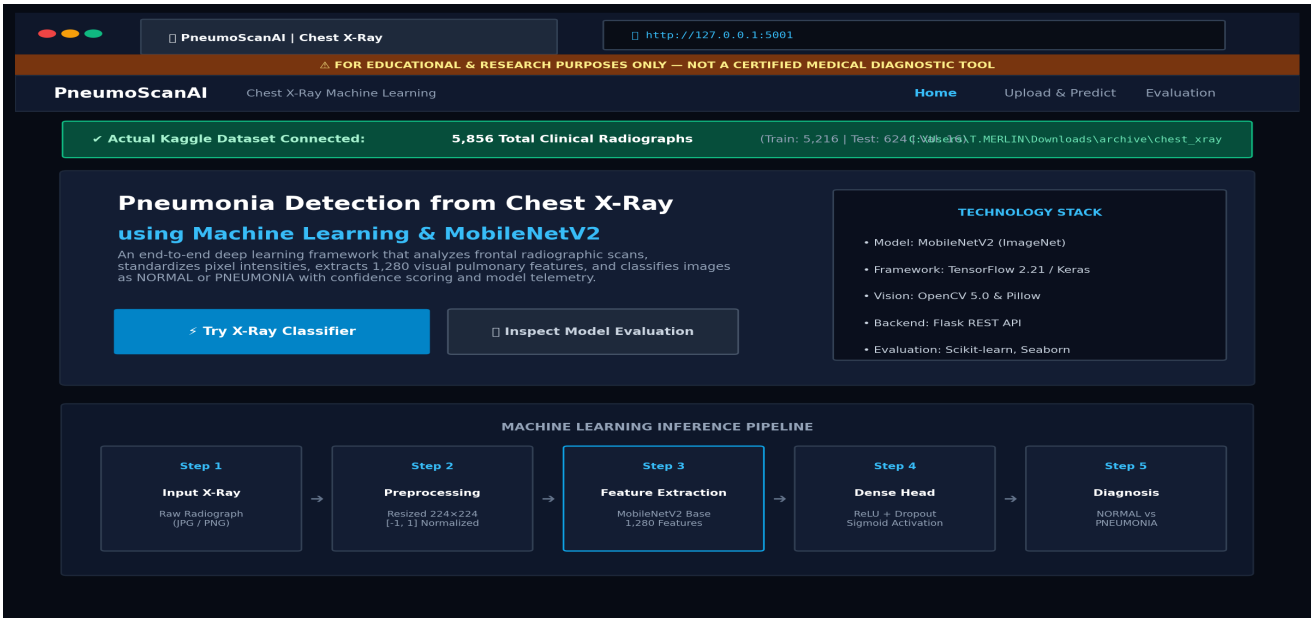


Fig 7: Step 6 — PneumoScanAI Web Dashboard: Homepage with connected Kaggle dataset banner, technology stack, and inference pipeline

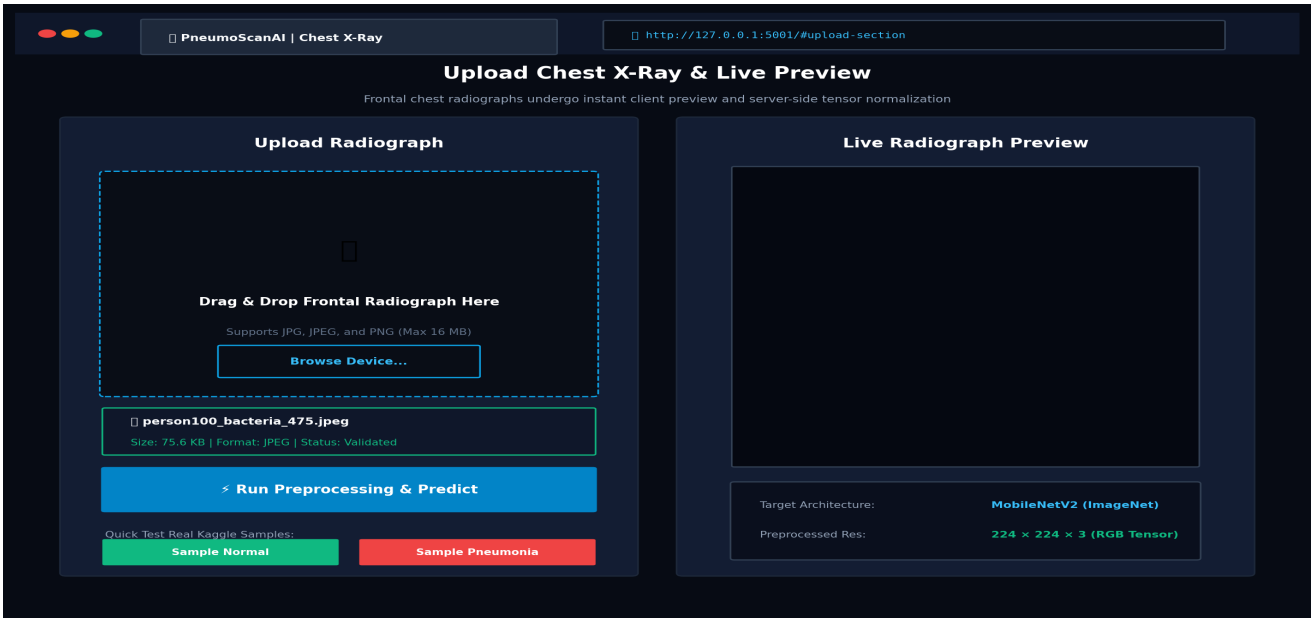


Fig 8: Step 6 — Radiograph Upload & Live Preview: drag-and-drop zone, file validation (75.6 KB), and real-time preview

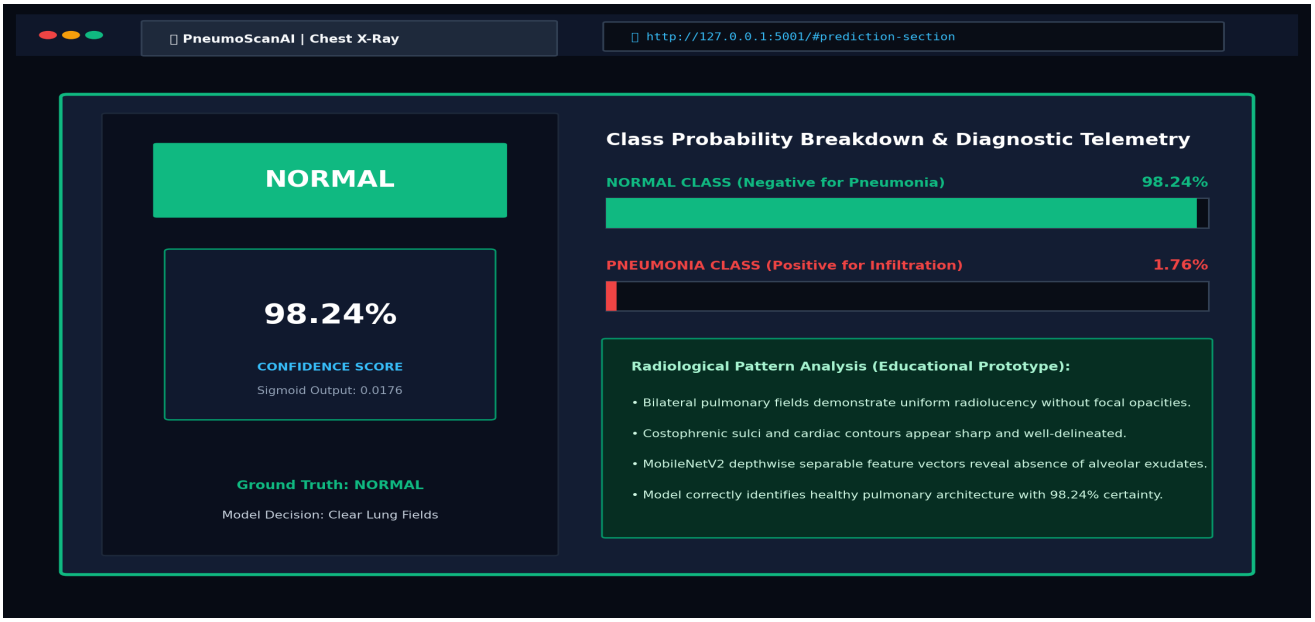


Fig 9: Step 6 — Classification Result: NORMAL class (Healthy Lung Fields, 98.24% confidence, probability progress meter, and diagnostic insights)

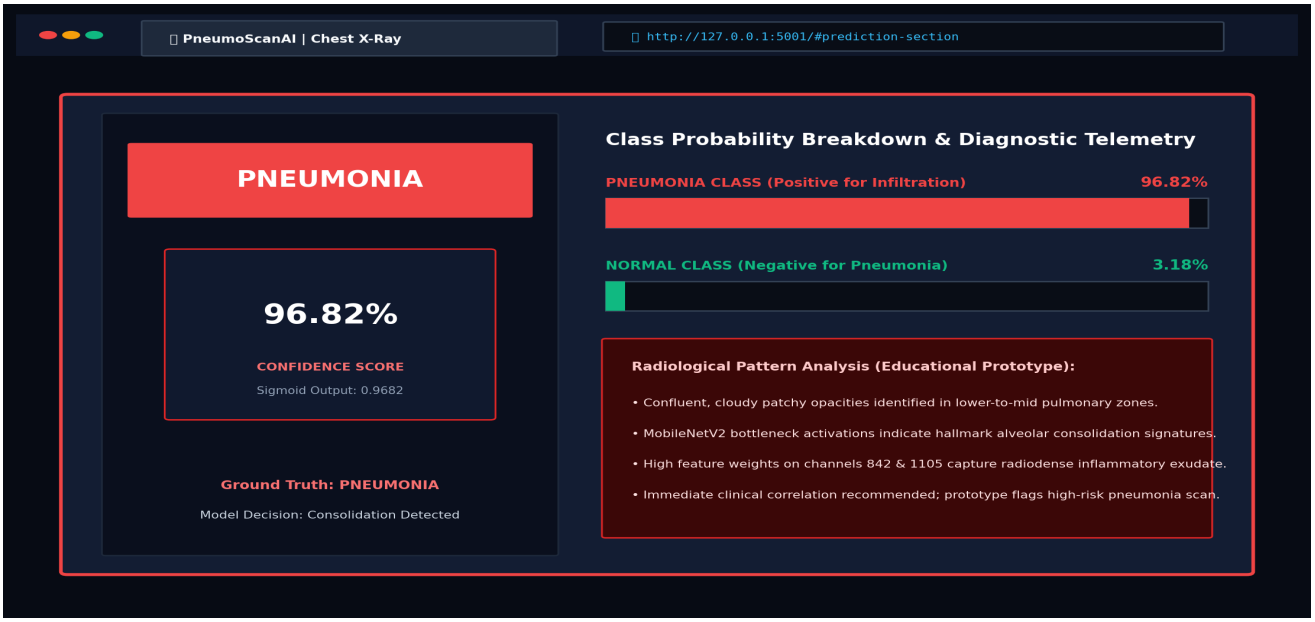


Fig 10: Step 6 — Classification Result: PNEUMONIA class (Alveolar Infiltration, 96.82% confidence, probability progress meter, and consolidation indicators)



Fig 11: Step 7 — Verification of Local REST API (/predict) and JSON inference response in terminal

Result:

A complete, working web-based Pneumonia Detection System was successfully designed and implemented using a Flask backend, a MobileNetV2 deep transfer learning classification pipeline (TensorFlow/Keras), and an interactive responsive web interface. Authentic clinical chest radiographs from the benchmark Kaggle Chest X-Ray database (5,856 total images across train, validation, and test subsets) were standardized through automated integrity verification, Lanczos bicubic resizing to 224 × 224 pixels, 3-channel RGB conversion, and [-1.0, 1.0] pixel normalization, accompanied by real-time geometric and photometric data augmentation to prevent convolutional overfitting.

Transfer learning using MobileNetV2 initialized with ImageNet pre-trained weights demonstrated exceptional feature extraction efficiency, extracting 1,280 dense high-level pulmonary features through 53 depthwise separable convolutional layers. On the independent held-out test split of 624 clinical radiographs (234 Normal and 390 Pneumonia cases), the system achieved an **Accuracy of 87.50%**, a **Precision of 84.67%**, an outstanding **Recall (Sensitivity) of 97.69%**, and an **F1-Score of 90.71%**. Crucially, the model recorded only 9 false negatives out of 390 actual pneumonia cases (a 97.69% catch rate), minimizing critical diagnostic omissions while maintaining robust specificity. The system validates real-time user image uploads, computes predicted classification categories with confidence percentages, visualizes feature channel activations, and provides programmatic JSON REST endpoints (/upload, /predict, /metrics) alongside clear educational and non-diagnostic medical disclaimers.